
Phase 5A - SQLite Logger

Step 1: Install SQLite

Verify:

None

```
sqlite3 --version
```

If not installed:

None

```
sudo apt install sqlite3
```

Step 2: Install Python MQTT package

None

```
pip3 install paho-mqtt
```

Step 3: Create logger file

None

```
nano vehicle_logger.py
```

Paste:

None

```
import sqlite3
import paho.mqtt.client as mqtt
import re
```

```
# -----  
# SQLite Setup  
# -----  
  
conn = sqlite3.connect("vehicle_telematics.db")  
cursor = conn.cursor()  
  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS location(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
    latitude REAL,  
    longitude REAL  
)  
""")  
  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS status(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
    speed INTEGER,  
    fuel INTEGER  
)  
""")  
  
cursor.execute("""  
CREATE TABLE IF NOT EXISTS diagnostics(  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,  
    rpm INTEGER,  
    engine_temp INTEGER  
)  
""")  
  
conn.commit()
```

```

# -----
# MQTT Callback
# -----

def on_message(client, userdata, msg):

    payload = msg.payload.decode()

    print(f"\nTopic: {msg.topic}")
    print(f"Payload: {payload}")

    if msg.topic == "vehicle/location":

        lat = float(
            re.search(
                r'Latitude=(\[d.\]+\)',
                payload).group(1)
        )

        lon = float(
            re.search(
                r'Longitude=(\[d.\]+\)',
                payload).group(1)
        )

        cursor.execute(
            "INSERT INTO location(latitude,longitude) VALUES(?,?)",
            (lat, lon)
        )

    elif msg.topic == "vehicle/status":

        speed = int(
            re.search(
                r'Speed=(\[d+\)',

```

```

        payload).group(1)
    )

    fuel = int(
        re.search(
            r'Fuel=(\d+)',
            payload).group(1)
    )

    cursor.execute(
        "INSERT INTO status(speed,fuel) VALUES(?,?)",
        (speed, fuel)
    )

elif msg.topic == "vehicle/diagnostics":

    rpm = int(
        re.search(
            r'RPM=(\d+)',
            payload).group(1)
    )

    temp = int(
        re.search(
            r'EngineTemp=(\d+)',
            payload).group(1)
    )

    cursor.execute(
        "INSERT INTO diagnostics(rpm,engine_temp) VALUES(?,?)",
        (rpm, temp)
    )

    conn.commit()

# -----

```

```
# MQTT Setup
# -----

client = mqtt.Client()

client.on_message = on_message

client.connect("localhost", 1883, 60)

client.subscribe("vehicle/location")
client.subscribe("vehicle/status")
client.subscribe("vehicle/diagnostics")

print("Vehicle Logger Started...")

client.loop_forever()
```

Save:

```
None
CTRL + O
ENTER
CTRL + X
```

Step 4: Run Logger

Terminal 1:

```
None
python3 vehicle_logger.py
```

```
aniruddhan@raspberrypi:~ $ nano vehicle_logger.py
aniruddhan@raspberrypi:~ $ python3 vehicle_logger.py
/home/aniruddhan/vehicle_logger.py:115: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  client = mqtt.Client()
Vehicle Logger Started...

Topic: vehicle/location
Payload: Latitude=13.0949 Longitude=80.2829

Topic: vehicle/status
Payload: Speed=59 Fuel=11

Topic: vehicle/diagnostics
Payload: RPM=692 EngineTemp=82

Topic: vehicle/location
Payload: Latitude=13.0951 Longitude=80.2831

Topic: vehicle/status
Payload: Speed=35 Fuel=13

Topic: vehicle/diagnostics
Payload: RPM=1896 EngineTemp=82

Topic: vehicle/location
Payload: Latitude=13.0953 Longitude=80.2833
```

Step 5: Run Publisher

Terminal 2:

```
None
./tcu_mqttCAN
```

Step 6: Verify Database

Open database in a separate Terminal:

```
None
sqlite3 vehicle telematics.db
```

Show tables:

```
None
.tables
```

Expected:

None

diagnostics
location
status

Check records:

None

```
SELECT * FROM location LIMIT 5;
```

None

```
SELECT * FROM status LIMIT 5;
```

None

```
SELECT * FROM diagnostics LIMIT 5;
```

```
aniruddhan@raspberrypi:~ $ sqlite3 vehicle telematics.db
SQLite version 3.40.1 2022-12-28 14:03:47
Enter ".help" for usage hints.
sqlite> .tables
diagnostics  location      status
sqlite> SELECT * FROM diagnostics LIMIT 5;
1|2026-06-12 18:34:17|692|82
2|2026-06-12 18:34:18|1896|82
3|2026-06-12 18:34:19|5342|89
4|2026-06-12 18:34:20|4911|80
5|2026-06-12 18:34:21|1284|88
sqlite> SELECT * FROM status LIMIT 7;
1|2026-06-12 18:34:17|59|11
2|2026-06-12 18:34:18|35|13
3|2026-06-12 18:34:19|97|79
4|2026-06-12 18:34:20|110|0
5|2026-06-12 18:34:21|21|86
6|2026-06-12 18:34:22|55|93
7|2026-06-12 18:34:23|61|7
sqlite> SELECT * FROM location LIMIT 3;
1|2026-06-12 18:34:17|13.0949|80.2829
2|2026-06-12 18:34:18|13.0951|80.2831
3|2026-06-12 18:34:19|13.0953|80.2833
```

